

Observation Task - 2

1. BASIC INFORMATION

Task No.: 02

Date: 16/09/2026

Title: Bank Account Management System using Classes and Objects

2. OBJECTIVE / AIM

To design and implement a Bank Account Management System in Java using classes and objects, demonstrating attributes, methods, constructors, object creation, inheritance, method overriding, polymorphism, deposits, withdrawals, interest calculation, fund transfers, and overdraft handling.

3. SYSTEM REQUIREMENTS

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM
- Sufficient storage space
- Keyboard and mouse

Software Requirements:

- Java Development Kit (JDK) 8 or above
- Java Runtime Environment (JRE)
- IntelliJ IDEA / Eclipse / NetBeans / Visual Studio Code

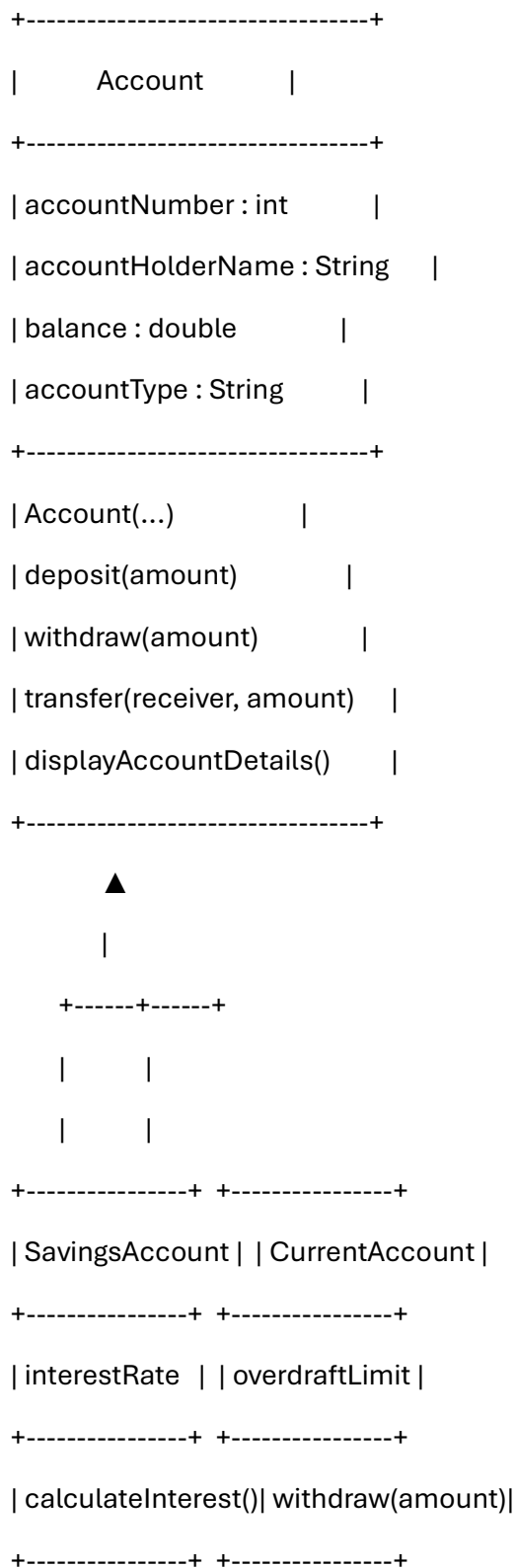
4. ALGORITHM / LOGIC DESCRIPTION

1. Create a base class named Account.
2. Define accountNumber, accountHolderName, balance, and accountType as attributes.
3. Create a constructor in the Account class to initialize the account details.
4. Define the deposit() method to add money to the account balance.
5. Define the withdraw() method to withdraw money when sufficient balance is available.
6. Define the transfer() method to transfer money from one account to another account.
7. Define the displayAccountDetails() method to display the account information.
8. Create a SavingsAccount class that inherits from the Account class.
9. Add interestRate as an additional attribute in SavingsAccount.
10. Define calculateInterest() to calculate interest based on the account balance.
11. Create a CurrentAccount class that inherits from the Account class.
12. Add overdraftLimit as an additional attribute in CurrentAccount.
13. Override the withdraw() method in CurrentAccount to allow withdrawal up to the permitted overdraft limit.
14. Create objects of SavingsAccount and CurrentAccount.
15. Display the account details before performing transactions.
16. Perform deposits, withdrawals, interest calculation, and fund transfer operations.
17. Display the account details after the transactions.
18. Display the final balance of both accounts.

OOP CONCEPTS DEMONSTRATED

- **Classes and Objects:** Account, SavingsAccount, and CurrentAccount are classes, and savings and current are objects.
- **Constructors:** Used to initialize account details.
- **Inheritance:** SavingsAccount and CurrentAccount inherit from Account.
- **Polymorphism:** Achieved through method overriding.
- **Method Overriding:** withdraw() is overridden in CurrentAccount.
- **super Keyword:** Used to call the constructor of the parent class.

5. UML CLASS DIAGRAM



6. SOURCE CODE

Java File Name: BankAccountManagement.java

```
class Account {  
    int accountNumber;  
    String accountHolderName;  
    double balance;  
    String accountType;  
  
    Account(int accountNumber, String accountHolderName,  
            double balance, String accountType) {  
        this.accountNumber = accountNumber;  
        this.accountHolderName = accountHolderName;  
        this.balance = balance;  
        this.accountType = accountType;  
    }  
  
    void deposit(double amount) {  
        balance = balance + amount;  
        System.out.println("Deposited: " + amount);  
    }  
  
    void withdraw(double amount) {  
        if (amount <= balance) {  
            balance = balance - amount;  
        }  
    }  
}
```

```
        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("Insufficient balance");
    }
}
```

```
void transfer(Account receiver, double amount) {
    if (amount <= balance) {
        balance = balance - amount;
        receiver.balance = receiver.balance + amount;
        System.out.println("Transferred: " + amount);
    } else {
        System.out.println("Insufficient balance for transfer");
    }
}
```

```
void displayAccountDetails() {
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Account Holder: " + accountHolderName);
    System.out.println("Account Type: " + accountType);
    System.out.println("Balance: " + balance);
}
}
```

```
class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(int accountNumber, String accountHolderName,
```

```

        double balance, double interestRate){
super(accountNumber, accountHolderName, balance, "Savings");
this.interestRate = interestRate;
}

void calculateInterest() {
    double interest = balance * interestRate / 100;
    balance = balance + interest;
    System.out.println("Interest Added: " + interest);
}
}

class CurrentAccount extends Account {
    double overdraftLimit;

    CurrentAccount(int accountNumber, String accountHolderName,
        double balance, double overdraftLimit) {
        super(accountNumber, accountHolderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Overdraft limit exceeded");
        }
    }
}

```

```
    }  
}  
}
```

```
public class BankAccountManagement {  
    public static void main(String[] args) {  
  
        SavingsAccount savings = new SavingsAccount(  
            101, "Rahul", 10000, 5);  
  
        CurrentAccount current = new CurrentAccount(  
            102, "Arjun", 5000, 3000);  
  
        System.out.println("BEFORE TRANSACTIONS");  
        savings.displayAccountDetails();  
        System.out.println();  
        current.displayAccountDetails();  
  
        System.out.println("\nTRANSACTIONS");  
  
        savings.deposit(2000);  
        savings.withdraw(1000);  
        savings.calculateInterest();  
  
        current.deposit(3000);  
        current.withdraw(9000);  
  
        savings.transfer(current, 2000);  
    }  
}
```

```
System.out.println("\nAFTER TRANSACTIONS");

savings.displayAccountDetails();

System.out.println();

current.displayAccountDetails();


System.out.println("\nFinal Balance of Savings Account: "
                    + savings.balance);


System.out.println("Final Balance of Current Account: "
                    + current.balance);
}
}
```

7. COMPILATION & EXECUTION COMMANDS

Java File Name:

BankAccountManagement.java

Compilation Command:

javac BankAccountManagement.java

Execution Command:

java BankAccountManagement

8. OUTPUT SCREEN

BEFORE TRANSACTIONS

Account Number: 101

Account Holder: Rahul

Account Type: Savings

Balance: 10000.0

Account Number: 102

Account Holder: Arjun

Account Type: Current

Balance: 5000.0

TRANSACTIONS

Deposited: 2000.0

Withdrawn: 1000.0

Interest Added: 550.0

Deposited: 3000.0

Withdrawn: 9000.0

Transferred: 2000.0

AFTER TRANSACTIONS

Account Number: 101

Account Holder: Rahul

Account Type: Savings

Balance: 9550.0

Account Number: 102

Account Holder: Arjun

Account Type: Current

Balance: 1000.0

Final Balance of Savings Account: 9550.0

Final Balance of Current Account: 1000.0

9. RESULT

The Bank Account Management System was successfully implemented in Java. The program demonstrates classes and objects, constructors, inheritance, polymorphism, method overriding, deposit, withdrawal, interest calculation, fund transfer, and overdraft handling.